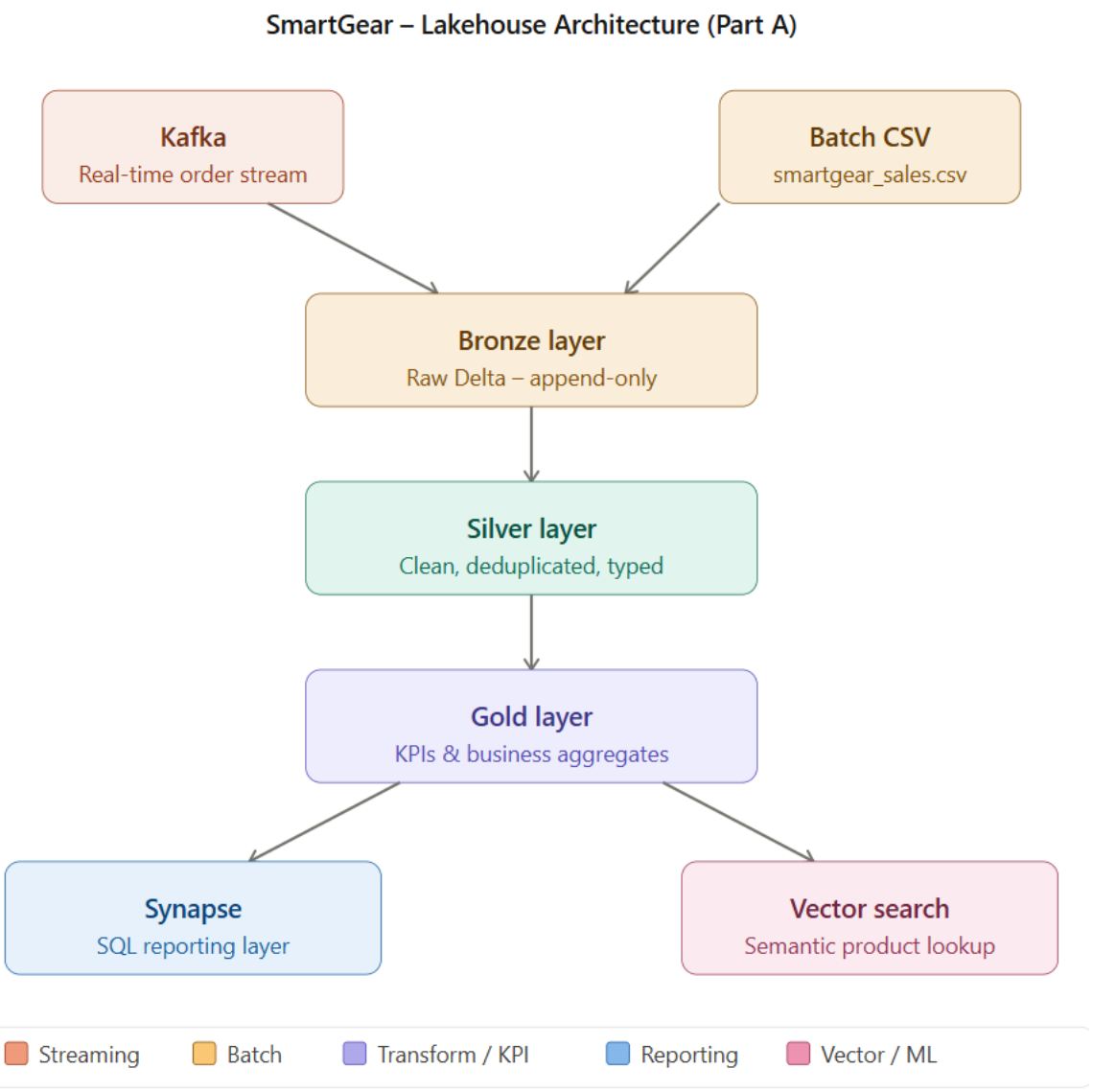
SmartGear Lakehouse Semantic Analytic Theory Answers

PART A – Architecture Diagram



Architecture Explanation

The data platform is designed using a Lakehouse architecture integrating streaming, batch, and semantic processing.

Kafka is used as the ingestion layer to capture real-time order data (smartgear_orders). Databricks processes this data using Structured Streaming and batch capabilities.

The Medallion architecture is implemented with Bronze, Silver, and Gold layers. Bronze stores raw data in Delta format, Silver performs cleaning and transformations, and Gold generates business KPIs and analytics.

Delta Lake ensures data reliability through ACID transactions, schema enforcement, and consistent data handling across streaming and batch workloads.

Batch data (smartgear_sales.csv) is used for validation and reconciliation of streaming outputs.

Synapse is used as the reporting layer where SQL queries are executed on Gold data for business insights.

A vector search layer is implemented using product metadata embeddings to enable semantic search, improving product discovery based on user intent.

Design Decisions

Why Lakehouse over Traditional Data Warehouse?

In this solution, data comes from different sources – Kafka (streaming), batch CSV, and product metadata for semantic search. A traditional data warehouse would require separate pipelines and data movement.

Using a Lakehouse (Databricks + Delta), all data is processed and stored in one platform. This avoids duplication, simplifies pipelines, and allows both real-time and analytical workloads to run on the same data.

Where Batch vs Streaming Fits?

Streaming is used for real-time order ingestion through Kafka, where data is continuously processed into Bronze → Silver → Gold layers.

Batch data (`smartgear_sales.csv`) is used for validation. For example, total revenue from streaming can be compared with batch data to identify mismatches or missing records.

Where Cost Trade-offs Arise?

Streaming requires continuous compute (Databricks jobs running frequently), which increases cost but provides near real-time insights.

Batch processing is cheaper since it runs periodically, but insights are delayed. In this implementation, micro-batching is used instead of continuous streaming to balance cost and performance.

Databricks vs Synapse

Spark in Databricks vs Synapse Spark

Databricks provides a fully managed and optimized Spark environment, which was used here for Kafka ingestion and transformations.

Synapse Spark exists but is typically used for lighter workloads. For this pipeline, Databricks is more suitable due to streaming and transformation requirements.

SQL Pools vs Databricks SQL

Databricks SQL is used to query Delta tables directly during development and analysis.

Synapse SQL Pools are designed for business users and reporting. In this solution, Gold layer data can be exposed to Synapse for dashboards and reporting.

When to Use Each in an Enterprise Setup

Databricks is used for building pipelines, handling streaming data, and performing transformations.

Synapse is used after data is prepared (Gold layer) for reporting, dashboards, and business queries.

Delta Lake Fundamentals

ACID in Data Lakes

Delta ensures that data is written reliably even during failures.

Example: If a streaming job fails while writing to Bronze, incomplete data is not saved, preventing corruption.

Schema Evolution

Schema evolution allows new fields to be added without breaking existing pipelines.

Example: If a new column like `discount` is introduced, it can be added without rewriting historical data.

Time Travel

Time travel allows access to previous versions of data.

Example: If incorrect data is written to Silver, the previous version can be restored.

Why Delta is Critical for Streaming Pipelines

Streaming pipelines continuously write data, so reliability is critical. Delta ensures:

- No duplicate writes
- Consistent schema
- Recovery from failures

Example: If the streaming job restarts, it continues from the last checkpoint without reprocessing data incorrectly.

PART B – Kafka + Streaming Pipeline

PART B1 – Kafka Theory

1. What is a Topic, Partition, Offset?

A **topic** is a logical channel in Kafka where messages are stored. For example, order data (`order_id`, `product`, `quantity`, `price`) is sent to the `smartgear_orders` topic.

A **partition** is a subdivision of a topic that enables parallel processing. The `smartgear_orders` topic is configured with **6 partitions**, allowing data to be processed concurrently.

An **offset** is a unique identifier for each message within a partition. For example, messages in a partition have offsets like 0, 1, 2, and so on. If a consumer has processed up to offset 2, it resumes from offset 3.

2. How does Kafka guarantee durability?

Kafka guarantees durability by writing messages to disk in a commit log and retaining them for a configured period. Data is also replicated across brokers to prevent loss.

For example, order messages in `smartgear_orders` remain available even if the consumer is temporarily down. In typical setups, data is retained for **a few days (e.g., ~7 days)** unless configured otherwise.

3. Difference between At-most once, At-least once, Exactly-once

- **At-most once**: Messages may be lost but are not duplicated. Used in log/metrics systems where occasional loss is acceptable.
- **At-least once**: No data loss, but duplicates may occur. Common in order/event processing where duplicates can be handled.
- **Exactly-once**: No loss and no duplication. Used in critical systems like billing or financial transactions.

For example, an order event:

- At-most once → order might be missed
- At-least once → order may be processed twice
- Exactly-once → order processed accurately once

4. What is consumer group rebalancing?

Consumer group rebalancing occurs when consumers join, leave, or fail. Kafka redistributes partitions among available consumers to ensure continuous processing.

For example, if a streaming job consuming `smartgear_orders` restarts, Kafka reassigns the partitions so data continues to be processed.

5. Why partitioning matters for scalability?

Partitioning enables parallel processing and improves throughput. Each partition can be consumed independently.

For example, with **6 partitions** in `smartgear_orders`, multiple consumers or processing tasks can read data simultaneously, improving performance.

PART D – Orchestration

D1 – Airflow Theory

Brief on Airflow

Airflow is an open-source workflow orchestration tool used to schedule, manage, and monitor data pipelines by defining workflows as Directed Acyclic Graphs (DAGs).

What is a DAG?

A Directed Acyclic Graph (DAG) represents a workflow in Airflow where tasks are defined with dependencies and executed in a specific order.

Example:

Kafka ingestion → Bronze load → Silver transformation → Gold aggregation

What is Idempotency?

Idempotency ensures that running a task multiple times produces the same result without duplication or inconsistency.

Example:

Re-running a Bronze load should not duplicate records.

What are Backfills?

Backfills allow execution of workflows for past dates to process historical data that may have been missed.

Example:

Running pipelines for previous days to load missing data.

What are Retries?

Retries allow failed tasks to be automatically re-executed based on configured retry policies.

Example:

Retry Kafka ingestion if connection fails.

What are Sensors?

Sensors are special tasks that wait for a specific condition before proceeding.

Example:

Wait until Kafka data is available before processing.

D2 – ADF Pipeline Design

Pipeline Overview

An Azure Data Factory (ADF) pipeline is designed to orchestrate the end-to-end data flow from Kafka ingestion to the Medallion layers (Bronze → Silver → Gold).

Pipeline Flow

```
Kafka → Bronze → Silver → Gold
```

- Kafka provides real-time streaming data
- Bronze layer stores raw data in Delta format
- Silver layer performs cleaning, deduplication, and transformation
- Gold layer generates business KPIs and aggregated insights

Pipeline Activities

Step	Activity
1	Trigger pipeline based on schedule or data availability
2	Execute Databricks notebook to ingest Kafka data into Bronze
3	Execute transformation notebook to process Silver layer
4	Execute aggregation notebook to generate Gold layer KPIs
5	Store final outputs for reporting (Delta / Synapse)

Trigger Strategy

- Scheduled trigger (e.g., every 5 minutes) for near real-time processing
- Event-based trigger when new data is available

Monitoring

- Monitor pipeline execution status in ADF
- Track success/failure and enable retries
- Validate data across layers using row counts and logs

Cost Considerations

- Use scheduled triggers instead of continuous execution to reduce cost
- Optimize Databricks cluster usage
- Avoid reprocessing already processed data

PART E – Streaming Design Patterns

Stateful vs Stateless Streaming

- **Stateless Streaming:** Each event is processed independently without using past data

  Example: Filtering orders where price > 1000

- **Stateful Streaming:** Processing depends on past data and maintains state

  Example: Calculating total revenue per region over time

What is Watermarking?

Watermarking defines how long the system should wait for late data before ignoring it.

Example: If watermark is 10 minutes, and an order arrives 15 minutes late, it will be ignored

Handling Late-Arriving Data

Late data is handled by:

- Allowing a delay window (watermark)
- Updating results if data arrives within that window

Example:

An order for 10:00 AM arrives at 10:05 AM → included

If it arrives at 10:20 AM → ignored

Exactly-Once in Spark + Kafka (How?)

Ensures each record is processed only once without duplication.

Achieved using:

- Kafka offsets (track processed messages)
- Spark checkpointing (store progress)
- Delta Lake (ensures reliable writes)

Example:

If a job fails after processing some records, it resumes from the last checkpoint and does not reprocess already completed data

PART F – Vector DB & Semantic Search

F1 – Theory

What are Embeddings?

Embeddings are numerical vector representations of data (such as text) that capture semantic meaning.

Items with similar meaning have similar vector representations.

Example:

“Running shoes” and “sports sneakers” will have similar embeddings even though the words are different.

Why Cosine Similarity Works?

Cosine similarity measures the angle between two vectors to determine how similar they are.

It is effective for embeddings because it focuses on direction (meaning) rather than magnitude.

Example:

If a user searches for “fitness watch”, cosine similarity will match it closely with “smartwatch for tracking fitness” because their meanings are similar.

Keyword Search vs Semantic Search

Keyword Search	Semantic Search
Matches exact words	Understands meaning
Literal matching	Context-aware
Fails for synonyms	Handles variations

Example:

- Keyword search: “laptop bag” → won’t match “notebook backpack”
- Semantic search: understands both refer to similar products

What is ANN (Approximate Nearest Neighbour)?

ANN is a technique used to quickly find similar vectors in large datasets.

It provides fast results by approximating the nearest matches instead of computing exact distances.

Example:

In an e-commerce system with millions of products, ANN helps quickly find products similar to “wireless headphones” without comparing against every item.

PART G – Mini Case

Scenario Understanding

- Revenue mismatch between batch and streaming

- Kafka lag spike
- Missing records in Gold layer

Possible Root Causes

- **Revenue mismatch**
  - Streaming job processed duplicate records (no proper deduplication using `order_id + timestamp`)
  - Batch and streaming using different time filters or time zones
- **Kafka lag spike**
  - Spark job not consuming data fast enough (insufficient resources or inefficient transformations)
  - Large micro-batch size or slow processing in Silver/Gold
- **Missing records in Gold**
  - Data dropped during deduplication or filtering in Silver layer
  - Failed Gold job or partial execution

How I Would Debug

- Compare record counts across layers:
  - Kafka → Bronze → Silver → Gold
- Check for duplicates in Bronze:
  - Validate using `order_id`
- Check Kafka lag using metrics (consumer lag)
- Review Spark job logs for failures or slow stages
- Validate aggregation logic in Gold (groupBy, filters)

Monitoring Design

- Track **Kafka consumer lag** continuously
- Monitor **row count changes** across layers (Bronze vs Silver vs Gold)
- Set alerts for:
  - Sudden drop in records
  - Spike in lag
  - Job failures

- Track **processing time per batch** to detect slow pipelines

Reflection (Cost vs Performance Trade-offs)

The implementation demonstrates the trade-offs between cost and performance in a modern data platform. Streaming pipelines using Kafka and Databricks enable near real-time insights but require continuous compute resources, making them more expensive. Batch processing, while cost-efficient, introduces latency and is better suited for validation and historical analysis.

The Medallion architecture helps structure data into Bronze, Silver, and Gold layers, improving performance and maintainability. Delta Lake ensures reliability through ACID transactions and schema enforcement, enabling consistent data processing across streaming and batch workloads.

Design decisions such as using micro-batch processing instead of continuous streaming help balance performance and cost. Additionally, integrating semantic search using vector embeddings enhances product discovery and adds an intelligent layer to the system.

Overall, the solution demonstrates a scalable and efficient architecture while managing trade-offs between latency, cost, and reliability.